
How Machines Learn to Improve

The Optimizer Course

Lesson 1 — The Map Before the Journey

Everything you need to understand *every* optimizer —
explained slowly, with pictures, and with real numbers

*Before we meet Adam, momentum, or any famous optimizer, we need a map.
This lesson hands you the five ideas that the entire rest of the course is built
on. No jargon dumped on you. We build each idea from a picture, then check
it with numbers you can follow on your fingers.*

1 Start here: what is an optimizer even *doing*?

Imagine you're dropped onto a giant hill, blindfolded, in thick fog. Your job is simple to say: **get to the lowest point**. You can't see the valley. You can't see where you started. All you can do is feel the ground right under your feet — is it sloping down to the left? Forward? — take a careful step that way, and feel again.

That's it. *That is what every optimizer in machine learning does.*

Optimization in one picture: get to the bottom of a hill you can't fully see



The “hill” is the model’s **loss** — a number that says how badly the model is doing. High up the hill = bad model. Down in the valley = good model. The “position” on the hill is the model’s **weights** (its tunable numbers). Training means: start somewhere random, feel the slope, step downhill, repeat — millions of times — until you reach the bottom.

The big idea

A model’s job is to fit data. An *optimizer*’s job is to find the weights that make it fit. The architecture is the car; the optimizer is how you drive it downhill. Every optimizer you’ll ever hear of is just a different answer to one question: **“which way do I step, and how big a step?”**

We’ll write the position as w (the weights) and the loss as $f(w)$. The whole game is:

find w that makes $f(w)$ as small as possible.

For a real network, w has *billions* of numbers and the fog never lifts — so we can only ever feel the local slope and step. This lesson is about learning to “feel” properly. There are exactly five things to feel, and we’ll take them one at a time.

1.1 Our practice hill

To keep everything concrete, we’ll use *one* small hill for the whole course, so you watch the same problem get solved by every optimizer. Here it is:

$$f(w) = \frac{1}{2} w^\top A w - b^\top w, \quad A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Say it in English: “the height of the hill at position w is some bowl shape (set by the matrix A) tilted by a slope (set by b).”

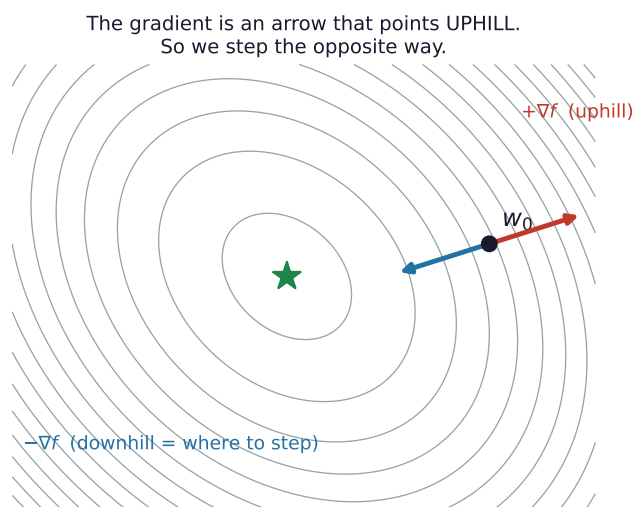
Don’t worry about *why* it’s written this way yet. A “bowl shape” (a quadratic) is the perfect practice hill because — as you’ll see in Section 5 — *every* smooth hill looks like a bowl when you zoom in close. Master the bowl and you’ve mastered the local behaviour of everything.

2 Feel #1 — The gradient: which way is downhill?

The first thing you feel with your feet is the **slope**. In math, the slope of a multi-dimensional hill is called the **gradient**, written $\nabla f(w)$. It’s just a list of “how steeply does the hill rise if I nudge each weight?”

The big idea

The gradient is an arrow. It points in the direction the hill rises *fastest* — i.e. **straight uphill**. So to go down, you step in the *opposite* direction, $-\nabla f$. That single sentence is the seed of every first-order optimizer (and that’s most of them).



For our bowl, there’s a clean formula for the gradient:

$$\nabla f(w) = A w - b.$$

Say it in English: “to find the slope, multiply your position by A and subtract b .” That’s the whole gradient. Let’s plug in a real point and see an actual arrow.

Let's actually do the numbers

Stand at the point $w_0 = (2, 1)$. The slope there is:

$$\nabla f(w_0) = Aw_0 - b = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 2 \end{bmatrix}.$$

Step by step, no skipping:

- Top row: $4 \times 2 + 1 \times 1 = 8 + 1 = 9$. Minus 1 gives **8**.
- Bottom row: $1 \times 2 + 3 \times 1 = 2 + 3 = 5$. Minus 2 gives **3**.

So $\nabla f(w_0) = (8, 3)$. The hill rises fastest in the direction $(8, 3)$, so the optimizer should step the opposite way, toward $(-8, -3)$. **That arrow is the entire instruction “go this way.”**

How do we know it points *exactly* uphill, not just roughly? Because the slope you'd feel walking in any direction u is the gradient “dotted” with that direction, $\nabla f \cdot u$, and that's biggest when u lines up with the gradient. (We'll use this fact constantly; for now just trust the picture: the red arrow is the steepest way up.)

2.1 Where's the bottom?

The bottom of the hill is where it's perfectly flat — the slope is zero in every direction. So we set $\nabla f = 0$, i.e. $Aw - b = 0$, and solve for w :

$$w^* = A^{-1}b \quad (\text{the exact bottom}).$$

Let's actually do the numbers

The inverse of $A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}$ is $\frac{1}{\det A} \begin{bmatrix} 3 & -1 \\ -1 & 4 \end{bmatrix}$, and $\det A = 4 \cdot 3 - 1 \cdot 1 = 11$. So:

$$w^* = \frac{1}{11} \begin{bmatrix} 3 & -1 \\ -1 & 4 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} = \frac{1}{11} \begin{bmatrix} 3 - 2 \\ -1 + 8 \end{bmatrix} = \frac{1}{11} \begin{bmatrix} 1 \\ 7 \end{bmatrix} \approx (0.091, 0.636).$$

Quick sanity check: plug w^* back in and the slope is $(0, 0)$ — perfectly flat. We're at the bottom. ✓

Watch out (common confusion)

We just *solved* for the bottom directly. Why not always do that?! Because for a real neural network there's no A^{-1} to compute — the hill isn't a clean bowl, and w has billions of entries. Solving $\nabla f = 0$ exactly is impossible. **That impossibility is the entire reason optimizers exist:** they crawl toward the bottom one step at a time instead of jumping there. On our toy bowl we happen to know the true answer, which is great — it lets us measure exactly how close each optimizer gets.

3 Feel #2 — The Hessian: is the ground curving sharply?

Knowing the slope isn't enough. Picture skiing: the slope tells you you're going downhill, but it doesn't warn you that a tight bend is coming. For that you need to feel how the slope is *changing*. That “change in slope” is the **curvature**, and the math object that holds it is the **Hessian**, written H — a grid of second derivatives.

The big idea

Gradient = slope (which way is down). Hessian = curvature (how sharply the ground bends). A sharp bend means a small step is safe but a big step overshoots. A gentle slope means you can stride confidently. **Almost every “smart” optimizer is secretly trying to estimate curvature so it can size its steps right.**

For our bowl, the curvature is beautifully simple — it’s just the matrix A again, the same everywhere:

$$H = A = \begin{bmatrix} 4 & 1 \\ 1 & 3 \end{bmatrix}.$$

(A bowl curves the same at every point; that’s what makes it a bowl. Real hills curve differently from place to place, which is what makes them hard.)

When you’re actually training

This curvature idea is the punchline of the whole course in advance. **Plain gradient descent ignores curvature** and pays for it by zig-zagging. **Adam, RMSProp, and friends** (Lessons 4–6) cheaply *estimate* curvature, one weight at a time, so they can take a bigger step where the ground is flat and a smaller one where it’s sharp. **Newton’s method** (Lesson 7) uses the *full* curvature and is the gold standard — but too expensive for big models. The entire field is a fight over “how much curvature can I afford to use?”

4 Feel #3 — The shape of the bowl, and why it makes life hard

Here’s the single most important practical idea in this lesson. Our bowl isn’t perfectly round — it’s a bit *stretched*, like an oval. Some directions are steep; others are shallow. We measure “how steep in each natural direction” using the **eigenvalues** of the curvature matrix.

Eigenvalue, in plain English: pick the bowl’s natural axes (its long axis and short axis). Along each axis, the steepness is just a single number — that number is an eigenvalue. Big eigenvalue = steep direction. Small eigenvalue = shallow direction.

Let’s actually do the numbers

The eigenvalues of A come from solving $\det(A - \lambda I) = 0$:

$$(4 - \lambda)(3 - \lambda) - 1 = \lambda^2 - 7\lambda + 11 = 0 \Rightarrow \lambda = \frac{7 \pm \sqrt{5}}{2}.$$

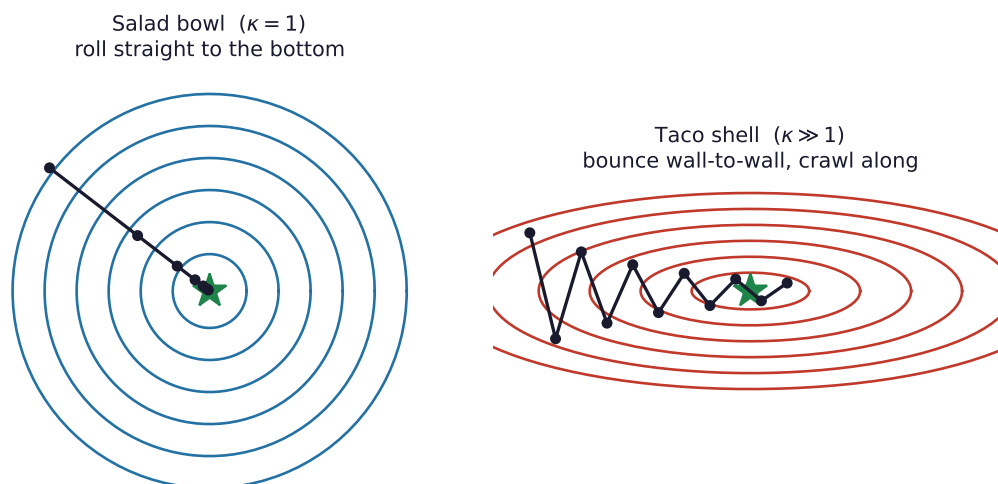
That gives a steep direction $\lambda_{\max} \approx 4.62$ and a shallow one $\lambda_{\min} \approx 2.38$. The bowl is about twice as steep one way as the other.

Now the key number. Divide the steepest by the shallowest:

$$\text{condition number } \kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \frac{4.62}{2.38} \approx 1.94.$$

Say it in English: “how lopsided is the bowl?” $\kappa = 1$ is a perfectly round salad bowl. Big κ is a long, thin taco shell.

Why should you care about lopsidedness? Because it decides how painful descent is:



In a round bowl, “downhill” points straight at the bottom — you roll right in. In a stretched taco, “downhill” mostly points at the *near wall*, not the far-away bottom. So you bounce wall-to-wall, inching along the length. **This zig-zag is the #1 enemy of basic gradient descent**, and beating it is the reason momentum, adaptive methods, and second-order methods were invented.

When you’re actually training

Our toy hill is gently oval ($\kappa \approx 1.94$). Real neural-network hills are absurdly lopsided — condition numbers in the *millions*. That’s a taco thousands of times longer than it is wide. If you remember one number from this lesson, remember κ : it predicts almost everything about how hard an optimizer has to work.

5 Feel #4 — The Taylor trick: turning any hill into a bowl

Time for the magic that ties the course together. We keep saying “every hill looks like a bowl up close.” Here’s why that’s literally true. If you stand at a point w_0 and look at the hill a small step d away, you can predict its height with the **Taylor expansion**:

$$f(w_0 + d) \approx \underbrace{f(w_0)}_{\text{height now}} + \underbrace{\nabla f(w_0)^\top d}_{\text{slope} \times \text{step}} + \underbrace{\frac{1}{2} d^\top H d}_{\text{curvature} \times \text{step}^2}.$$

Say it in English: “new height = current height, plus how far you stepped along the slope, plus a curvature correction for stepping too far.” The first two pieces are a flat ramp; adding the third piece bends that ramp into a bowl.

The big idea

Use only the slope term $\Rightarrow$ you’re approximating the hill as a flat ramp. Step against it and you’ve got **gradient descent** (and SGD, momentum, Adam — all the first-order methods). Add the curvature term $\Rightarrow$ you’ve got a little bowl, and jumping to its bottom is **Newton’s method**. *Every optimizer in this course is a choice about how much of this expansion to use.*

Let's actually do the numbers

Let's prove the formula actually works. Stand at $w_0 = (2, 1)$ where $f(w_0) = 7.5$ and $\nabla f(w_0) = (8, 3)$. Take a small step $d = (-0.3, 0.2)$.

- **Slope piece:** $\nabla f \cdot d = 8(-0.3) + 3(0.2) = -2.4 + 0.6 = -1.8$. (Negative — good, we went downhill.)
- **Curvature piece:** first $Ad = (4(-0.3) + 1(0.2), 1(-0.3) + 3(0.2)) = (-1.0, 0.3)$, then $\frac{1}{2} d \cdot (Ad) = \frac{1}{2}((-0.3)(-1.0) + (0.2)(0.3)) = \frac{1}{2}(0.36) = 0.18$.
- **Prediction:** $7.5 - 1.8 + 0.18 = 5.88$.

Now compute the *true* height $f(1.7, 1.2)$ directly $\rightarrow$ it's exactly **5.88**. The prediction nailed it. (For a real curvy hill it'd be *approximately* right for small steps — which is exactly why optimizers take small steps.)

Notice the tug-of-war: the slope piece (-1.8) *wants* you to step far; the curvature piece ($+0.18$) *punishes* stepping too far. The **learning rate** (Lesson 2) is the knob that settles this fight.

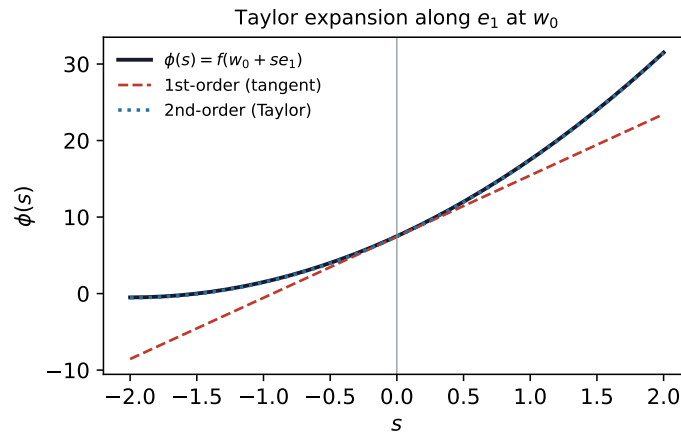
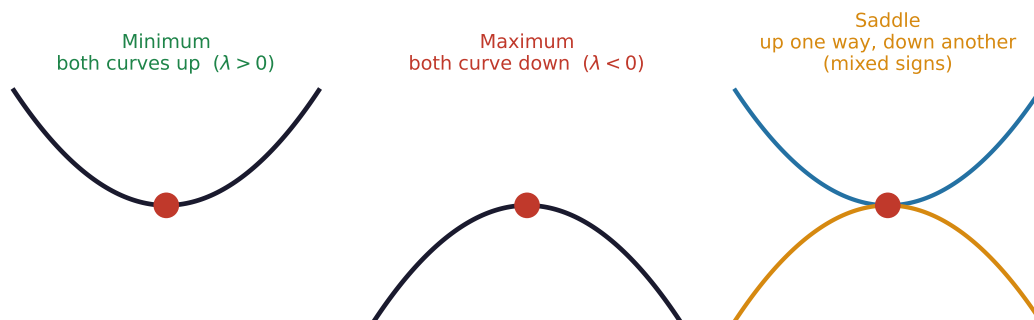


Figure 1: Slicing the hill along one axis. The true hill (black) is a curve. A “flat ramp” guess (red) is only good for tiny steps. Adding curvature (blue dotted) matches the real curve. First-order optimizers trust the red line, so they tiptoe; second-order optimizers trust the blue curve, so they can leap.

6 Feel #5 — Bottoms, tops, and sneaky saddles

When the slope is zero, you've stopped — but stopped *where*? There are three possibilities, and you tell them apart using the eigenvalues (the steepness numbers) of the curvature:

- **Minimum** — curves *up* in every direction (all eigenvalues positive). The real bottom. What we want.
- **Maximum** — curves *down* everywhere (all negative). A hilltop. We never want this, and we basically never get stuck here.
- **Saddle** — curves up some ways, down others (*mixed* signs). Like a horse saddle or a mountain pass. **This is the real villain in deep learning.**



When you're actually training

Why obsess over saddles? Because in billion-dimensional space, almost every flat spot is a saddle, not a true minimum. The slope is zero, so naive optimizers *think* they're done and freeze — even though there's a perfectly good “downhill” direction available (the one with negative curvature). A huge amount of optimizer design (momentum, noise in SGD, second-order methods) is really about **not getting fooled by saddles**.

A 10-second example: the hill $g(x, y) = x^2 - y^2$ has zero slope at the origin, but its curvature is $+2$ along x and -2 along y — mixed signs — so it's a saddle. Up if you walk east, down if you walk north. Stand exactly at the centre and you feel flat, but you're not at any bottom.

7 Putting it together: the map for the whole course

Take a breath — you now have all five feels. Watch how they unify *everything* coming. At any spot, an optimizer can build the little local bowl from Feel #4:

$$\hat{f}(d) = f(w_0) + \underbrace{\nabla f(w_0)^\top d}_{\text{slope}} + \underbrace{\frac{1}{2} d^\top H d}_{\text{curvature}},$$

and each famous optimizer is just a different decision about what to do with it:

Lesson	Optimizers	Their one-line trick
2. The basic loop	GD, SGD, minibatch	step against the slope
3. Adding memory	Momentum, Nesterov	remember past steps, build speed
4. Per-parameter steps	Adagrad, Adadelata, RMSProp	cheap curvature guess, one weight at a time
5. The modern default	Adam, AdamW, Nadam, RAdam, ...	momentum + per-weight scaling + fixes
6. Training giants	AdaFactor, LAMB, LARS, NovoGrad	the same, but memory-cheap and huge-batch
7. Using curvature	Newton, L-BFGS, KFAC, Shampoo, Sophia	actually use the Hessian (cleverly approximated)
8. Generalizing well	SAM, Lookahead	aim for <i>flat</i> bottoms, not just low ones
9. Schedules & no-tuning	warmup, cosine, ScheduleFree, Prodigy	how to change the step size over time
10. Frontier & choosing	Lion, Muon	the newest tricks, and how to pick

It's all one idea, viewed from sharper and sharper angles: *how much curvature can I afford, and how do I guess it cheaply?* Hold this table in your head. Every lesson fills in one row — with the same pictures, the same plain English, and the same kind of real-number walkthroughs you just did.

The big idea

The five feels, one last time: (1) *Gradient* — which way is downhill. (2) *Hessian* — how sharply the ground bends. (3) *Condition number* — how lopsided the bowl is (the source of zig-zag pain). (4) *Taylor* — any hill is a bowl up close, so we can plan a step. (5) *Min/max/saddle* — knowing what kind of flat spot you've hit. Get these, and every optimizer becomes obvious instead of mysterious.

Next — Lesson 2: The Basic Loop. We turn Feel #1 into the simplest real optimizer: “feel slope, step downhill, repeat.” We'll run it by hand on this exact hill, watch it zig-zag because the bowl is lopsided, figure out the biggest safe step size, and then push the step too far and watch it blow up — all with numbers you can check yourself.